Write specs that make AI deliver results with GitHub Copilot

Description

In this hands-on lab, you will work as an AI-assisted software professional using GitHub Copilot and the FocusSync application.

You will begin with a deliberately vague feature request and use GitHub Copilot's Ask and Plan capabilities to explore the hidden assumptions, implementation considerations, risks, constraints, and acceptance criteria that sit beneath the requirement. Rather than moving directly to code generation, you will learn how to use AI to clarify intent and reduce ambiguity before implementation begins

During the lab you will:

- Analyse a new feature request within an existing application.
- Use GitHub Copilot to identify unanswered questions and implementation assumptions.
- Convert AI discussion outputs into a structured specification.
- Define users, constraints, failure conditions, and out-of-scope boundaries.
- Review and refine AI-generated implementation plans.
- Create testable acceptance criteria using behaviour-focused specifications.
- Generate and evaluate test scenarios before implementation starts.

By the end of the exercise, you will have produced a complete AI-ready feature specification that is relevant, precise, unambiguous, and testable, giving an AI coding agent significantly clearer instructions while reducing unnecessary assumptions and rework.

Expected Output

A documented feature specification containing:

- Goal and business intent
- User requirements
- Functional requirements
- Constraints and assumptions
- Out-of-scope definitions
- Acceptance criteria
- Test scenarios
- AI-reviewed implementation plan

Success Criteria

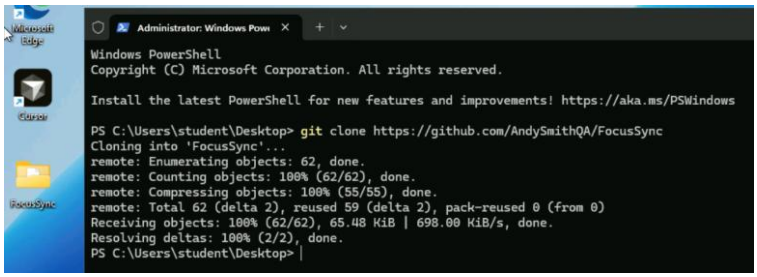
You can demonstrate that your specification:

- Minimises ambiguity.
- Explicitly defines important implementation decisions.
- Includes measurable and testable outcomes.
- Provides sufficient information for an AI coding agent to produce an implementation plan with minimal clarification requests

Step 1 – Open the FosucSync App

We have prepared an application for you to extend using the techniques in the course

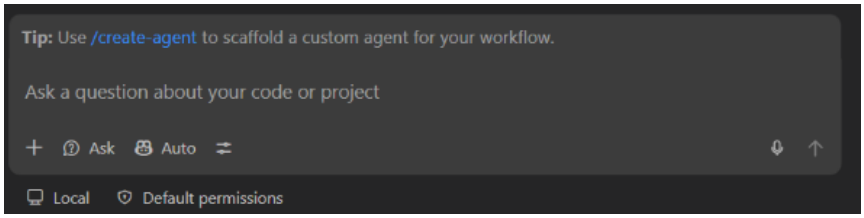
git clone https://github.com/AndySmithQA/FocusSync



Open VSCode and select the FocusSync Folder or right click the folder and select 'Open with Code'

Step 2 – Ask

Ensure that you have selected the agent chat window and have selected 'Ask' mode



The first step is to have a conversation with the agent about what you want to build. This is read only and changes no files at all.

Your challenge is to create a new chat feature that people who join the FocusSync sessions can use. for example:

I want to create a new feature where delegates can chat with each other while in session. What do I need to consider?

You will take the output as guidance to write the spec for the plan section.

Step 3 – Plan

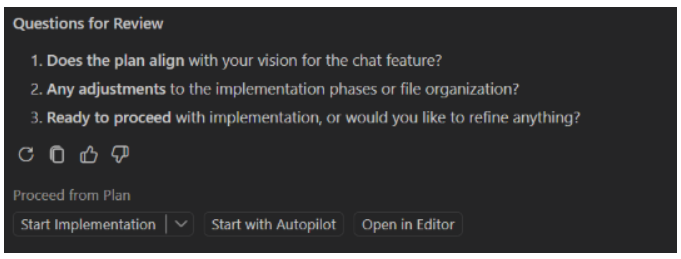
Once you have gotten all the ideas and answers from the Ask conversation, you need to plan.

Switch to Plan mode:

If you are struggling to think of something to prompt, have a look at some of these suggestions.

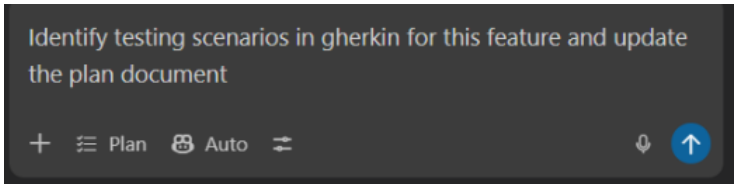
- Implement room-based chat for FocusSync using the existing room-scoped WebSocket architecture.
- Add chat_message (client), chat_event and chat_history (server) types in shared/types.ts.
- Store messages in a per-room in-memory ring buffer capped at 100 messages and send a history snapshot once when a user joins. Handle chat in server/src/ws/handler.ts and broadcast new messages using broadcastToRoom() without rebroadcasting history.
- Validate that senders belong to the room, reject empty messages, limit messages to 500 characters, sanitise to plain text, and rate-limit to 5 messages per 10 seconds per participant.
- Create a dedicated client chat store/reducer rather than embedding chat in room_state.
- Add a simple chat panel to RoomPage with message list, input box, Enter-to-send, unread badge, and collapsed-by-default behaviour so the timer remains the primary UI.
- Store messages against participantId and resolve display names from current room participants.
- Chat must be fully ephemeral and deleted when the room expires, empties, or the server restarts.
- Do not implement persistence, accounts, DMs, reactions, typing indicators, moderation UI, file uploads, edit/delete, or notifications beyond unread badges.

Make any edits you want to the live doc. Treat it like a PR review.

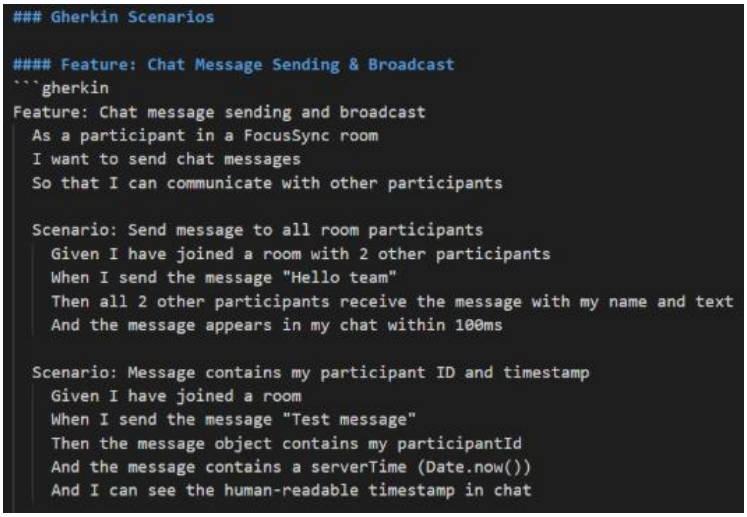


Step 4 – Refine Specs to produce more testable outcomes

Still in plan mode, prompt GitHub to identify testing scenarios



This will update the plan document



```
### Gherkin Scenarios

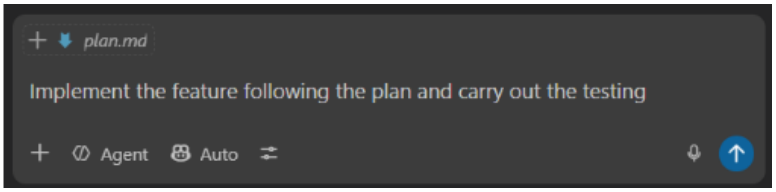
#### Feature: Chat Message Sending & Broadcast
```gherkin
Feature: Chat message sending and broadcast
  As a participant in a FocusSync room
  I want to send chat messages
  So that I can communicate with other participants

  Scenario: Send message to all room participants
    Given I have joined a room with 2 other participants
    When I send the message "Hello team"
    Then all 2 other participants receive the message with my name and text
    And the message appears in my chat within 100ms

  Scenario: Message contains my participant ID and timestamp
    Given I have joined a room
    When I send the message "Test message"
    Then the message object contains my participantId
    And the message contains a serverTime (Date.now())
    And I can see the human-readable timestamp in chat
```

Step 5 – Implement

You can now switch to Agent mode and finalise the build



Once it has finished, spin your creation up and test it!

If you spot anything that doesnt work as expected, you can drop the question directly into the chat window and let it suggest a solution.

Happy Coding!
```